# CS 216: Bitcoin Transaction Lab Assignment Report

---


## Team Hyperledgerz :

Disha Umesh Dange - 240001026
Rida Samrin - 240001060
Deo Shriya Shamsunder - 240041013
Siddhi Sandesh Patil - 240041035


---

# Part 1:Legacy Address Transactions (P2PKH)

Within this section, the complete process of creating, signing, and validating the Bitcoin Legacy P2PKH transactions is explained using the local regtest environment. The main goal is to show the underlying cryptographic link between the "Challenge" scripts (ScriptPubKey) and the "Response" scripts (ScriptSig). The Python-based RPC calls to the underlying Bitcoin Core daemon (bitcoind) simulate the entire process from the initial funding to the final unlocking using the Python language and the RPC protocol. This section is concluded with the technical explanation of the underlying Bitcoin Script Execution Stack, which is verified using the manual debugger to confirm the underlying mathematics of the transaction logic.

## Workflow & Transaction Analysis :-

### Transaction Chain Overview :

The workflow implements a sequential chain of ownership, demonstrating how Bitcoin is spent by referencing previous outputs.

- Funding Address A: Address A is funded via coinbase maturation (mining blocks until the coinbase maturity was reached).
- Transaction A → B: A raw transaction is constructed to send 10.0 BTC to Address B. This transaction creates the Challenge that Address B must later solve.
- Transaction B → C: Address B identifies the previous transaction's output as an Unspent Transaction Output (UTXO). It provides the Response (Signature and Public Key) to unlock the funds and transfer them to Address C.

### Transaction Identifiers (TXIDs) :

The following unique transaction IDs were generated during execution:

- **TXID (A → B) :** 29af72df9dad78f91df25ffcd9eded65a20dd922c770a7152ffdcf266030caf6
- **TXID (B → C) :** 35d94a4c3b07d62b49b4cc651043f311ac894ee4e485b31c016e330dd27b0492

This output captures the funding of Address A via coinbase maturation and the subsequent transfer of 10.0 BTC to Address B. The ScriptPubKey (Challenge) for Address B is established here, and the resulting TXID is identified as the UTXO required for the next transaction in the chain.

```
Wallet 'testwallet' is already active.
Address A: myJGTs8qFYh1k9E8qV4KhmQVqmAStGXB7u
Address B: mtCmQgexgBnNzkXZQWWGro2EyYMXLkb3Yp
Address C: mt3ofsYistrfxmzFryoWaEU6A28kUNfpja
Wallet balance: 37.49987750 BTC

--- ScriptPubKey for Address B ---
OP_DUP OP_HASH160 8b290f9b978ec1f87a38c004ad457262bb4c91c4 OP_EQUALVERIFY OP_CHECKSIG

Transaction signed successfully.
Transaction broadcasted! TXID: 29af72df9dad78f91df25ffcd9eded65a20dd922c770a7152ffdcf266030caf6

--- Finding UTXO for Address B ---
UTXO from B:
TXID: 29af72df9dad78f91df25ffcd9eded65a20dd922c770a7152ffdcf266030caf6
Amount: 10.00000000
```

The below output shows the final stage in the workflow for P2PKH transactions. In this output, we can see how the signed transaction is decoded where the ScriptSig (Response) is generated for the unlocking script, which contains the ECDSA signature and Public Key required for satisfying Address B's previous challenge. We can also see how a new ScriptPubKey (Locking Script) for Address C is generated successfully.

```
--- Creating Transaction B -> C ---
Raw transaction B -> C created.

--- Decoding Raw Transaction B -> C ---

ScriptPubKey for C:
OP_DUP OP_HASH160 897734962996266cac2ca2e1af366a1d904cd22b OP_EQUALVERIFY OP_CHECKSIG

--- Signing Transaction ---
Signed transaction:
Transaction B -> C signed successfully.

--- Broadcasting Transaction ---
Transaction B -> C broadcasted!
TXID: 35d94a4c3b07d62b49b4cc651043f311ac894ee4e485b31c016e330dd27b0492

--- Mining a block to confirm transaction ---

--- Decoding Final Transaction ---
Transaction decoded successfully.

--- ScriptSig Analysis ---
Unlocking Script (scriptSig): 304402200512b387913ce26e455b27adb41b4430897d97400640da582d1653fab23f317b0220060e15c0e6d12062c3a3ad2c45101f76e80bb97eda
b360107bc29509a7679871[ALL] 03bfee9fcd44fe48969dd90846702d802e3d92296bd4164540398634e1b18b5e61

--- ScriptPubKey Analysis ---
Locking Script (scriptPubKey): OP_DUP OP_HASH160 897734962996266cac2ca2e1af366a1d904cd22b OP_EQUALVERIFY OP_CHECKSIG
```

## Script Analysis (Challenge vs. Response) :

### The Challenge: ScriptPubKey (Locking Script) :

The locking script for Address B acts as the cryptographic puzzle.

- **ASM Code:** OP_DUP OP_HASH160 8b290f9b978ec1f87a38c004ad457262bb4c91c4 OP_EQUALVERIFY OP_CHECKSIG
- **Mechanism:** This script requires the spender to provide a Public Key that, when hashed (Hash160), matches the value 8b29...91c4, followed by a signature that proves ownership of that key.

### The Response: ScriptSig (Unlocking Script) :

The unlocking script provided in the transaction from B → C serves as the solution to the previous challenge.

- **Unlocking Script:** 304402200512b387913ce26e455b27adb41b4430897d97400640da582d1653fab23f317b0220060e15c0e6d12062c3a3ad2c45101f76e80bb97edab360107bc29509a7679871[ALL] 03bfee9fcd44fe48969dd90846702d802e3d92296bd4164540398634e1b18b5e61
- **Structure:** It consists of two data pushes:
    1. **Digital Signature (ECDSA):** 304402200512b387913ce26e455b27adb41b4430897d97400640da582d1653fab23f317b0220060e15c0e6d12062c3a3ad2c45101f76e80bb97edab360107bc29509a7679871
    2. **Public Key:** 03bfee9fcd44fe48969dd90846702d802e3d92296bd4164540398634e1b18b5e61

The Public Key provided in the Response (ScriptSig) hashes to `8b290f...`, which matches the requirement of the Challenge (ScriptPubKey) exactly, confirming the link between the two transactions.

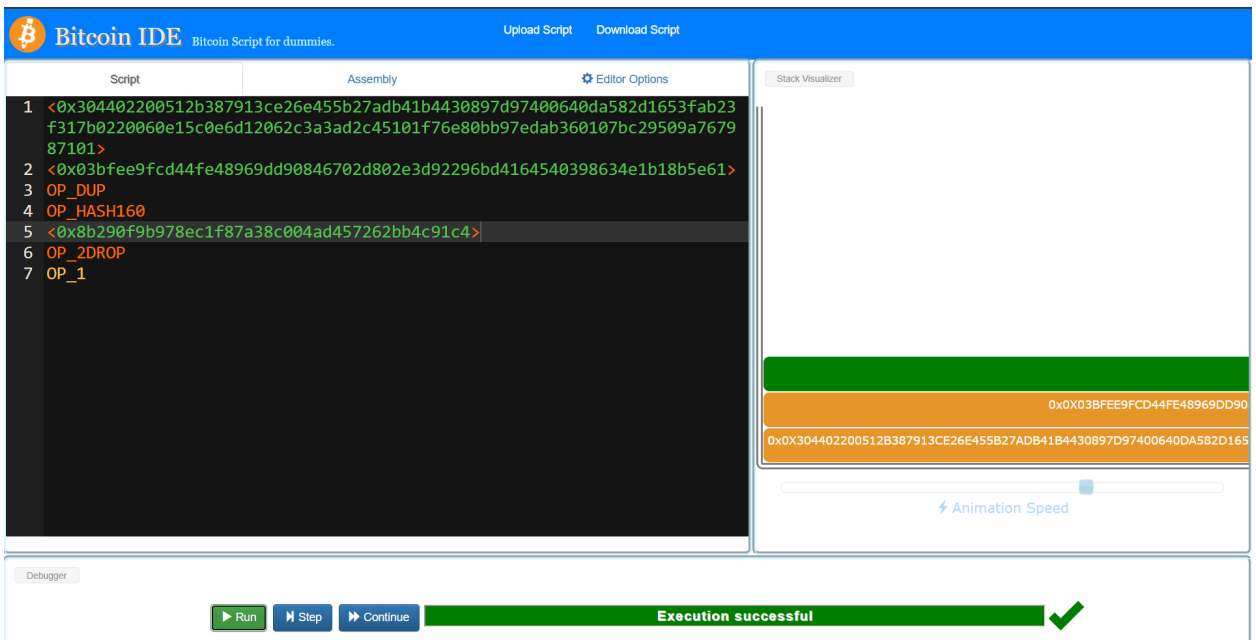
## Bitcoin Debugger Validation :

To ensure correctness, the scripts were validated using a Bitcoin Script IDE. The stack execution confirms the validity of the P2PKH logic.

## Step-by-Step Validation

1. **Push:** The signature and public key are pushed onto the stack.
2. **Duplicate:** `OP_DUP` creates a copy of the public key.
3. **Hash:** `OP_HASH160` hashes the copy to generate a 20-byte hash.
4. **Verify:** `OP_EQUALVERIFY` compares the generated hash to the address hash `8b29...91c4`.
5. **Signature Check:** `OP_CHECKSIG` validates the signature against the public key and transaction data.

---




---

In the standalone Bitcoin IDE, OP_2DROP and OP_1 were used to finalize the stack. This is because standalone debuggers lack the full transaction context required for OP_CHECKSIG to mathematically verify the ECDSA signature, though the preceding logic (OP_EQUALVERIFY) confirms the correct public key was provided.

# Part 2: P2SH-SegWit Address Transactions

## 1. Objective :

The objective of this part is to understand how **P2SH-P2WPKH (SegWit) transactions** are created and validated using Bitcoin Core in **regtest mode**. The program connects to the Bitcoin node using RPC, generates SegWit addresses, creates transactions, signs them using wallet keys, broadcasts them to the network, and analyzes the scripts involved in the validation process

## 2. Generating P2SH-SegWit Addresses :

The program first generates three P2SH-SegWit addresses.

```
Address A': 2Mvbtm7pjLqAmRZMHKvu6HfZLr1D8uN2Lhq
Address B': 2NBTskfVGSVqu3UhCLHy77nBA8srxfMbdFP
Address C': 2MtqPbU5BEoopfnJsUseTUsBrnHqM6qEgkE
```

These addresses use P2SH-wrapped SegWit, meaning SegWit functionality is embedded inside a P2SH address format.Unlike traditional legacy Bitcoin addresses, P2SH-SegWit addresses wrap a **SegWit redeem script inside a P2SH structure**. This design was introduced to allow SegWit transactions to remain compatible with older wallets and nodes that do not fully support native SegWit formats.

In this format, the address actually represents the **hash of a redeem script**, rather than a direct public key hash. The redeem script internally contains a **SegWit script (P2WPKH)** which handles the witness data used for validating the transaction signatures.

## Step 3: Fund Address A′ :

After generating the SegWit addresses, the next step is to provide funds to **Address A′** so that it can be used to create transactions. This is done using the `sendtoaddress` RPC command, which transfers **10 BTC** from the wallet to the generated P2SH-SegWit address A′.

```
Address A': 2Mvbtm7pjLqAmRZMHKvu6HfZLr1D8uN2Lhq
Address B': 2NBTskfVGSVqu3UhCLHy77nBA8srxfMbdFP
Address C': 2MtqPbU5BEoopfnJsUseTUsBrnHqM6qEgkE
Funding TXID: 9b0403ba228b643ca482fe39d42947e956d2a12bf7e7935426a064023d6f0ecd
UTXO for A': 9b0403ba228b643ca482fe39d42947e956d2a12bf7e7935426a064023d6f0ecd 0 10.00000000
```

A block is mined to confirm this transaction, ensuring that the funds become available as a UTXO (Unspent Transaction Output).

## Step 4: Create Transaction A′ → B′ :

The program retrieves the UTXO belonging to address A′ using listunspent.

The retrieved UTXO contains important information such as the **transaction ID (TXID), output index (vout), and the amount of BTC available**. This information is required to construct a new transaction. Using the `createrawtransaction` command, a raw transaction is created that uses the UTXO from address A′ as the input and sends the funds to address **B′** as the output. A small amount of BTC is subtracted as a transaction fee.

Once the raw transaction is created, it is decoded using the `decoderawtransaction` command. Decoding the transaction allows us to inspect the transaction structure, including inputs, outputs, and the associated scripts.

```
Raw Transaction A'->B': 0200000001cd0e6f3d0264a0265493e7f72ba1d256e94729d439fe82a43c648b22ba03049b0000000000fdffffff01f0a29a3b0000000017a914c7d6c7ca883c68ad71
8c32cece5125b9db4a8ae38700000000

Decoded Transaction (Challenge Script B'):
 {'txid': 'ec903ab3e756d1a683dae07416fab3da1b4bcf85e9792d477f85464758e83208', 'hash': 'ec903ab3e756d1a683dae07416fab3da1b4bcf85e9792d477f85464758e83208', 'ver
sion': 2, 'size': 83, 'vsize': 83, 'weight': 332, 'locktime': 0, 'vin': [{'txid': '9b0403ba228b643ca482fe39d42947e956d2a12bf7e7935426a064023d6f0ecd', 'vout':
0, 'scriptSig': {'asm': '', 'hex': ''}, 'sequence': 4294967293}], 'vout': [{'value': Decimal('9.99990000'), 'n': 0, 'scriptPubKey': {'asm': 'OP_HASH160 c7d6c7
ca883c68ad718c32cece5125b9db4a8ae3 OP_EQUAL', 'desc': 'addr(2NBTskfVGSVqu3UhCLHy77nBA8srxfMbdFP)#hjdpcarn', 'hex': 'a914c7d6c7ca883c68ad718c32cece5125b9db4a8a
e387', 'address': '2NBTskfVGSVqu3UhCLHy77nBA8srxfMbdFP', 'type': 'scripthash'}}]}
```

The decoded output shows the **locking script (scriptPubKey)** for address B′. Since B′ is a P2SH-SegWit address, the locking script follows the P2SH format:

‘OP_HASH160 <scriptHash> OP_EQUAL’

## Step 5: Sign and Broadcast Transaction A′ → B′ :

The raw transaction is signed using the wallet’s private key with the `signrawtransactionwithwallet` command.

The signature proves that the owner of address A′ has authorized the spending of the UTXO associated with that address. In the case of **P2SH-SegWit transactions**, the signature and public key are placed in the **witness data**, which is stored separately from the main transaction structure. This separation of witness data is one of the key features of SegWit and helps reduce transaction size and improve network efficiency.

Once the transaction is successfully signed, it becomes a **complete and valid transaction** that can be propagated through the network. The signed transaction is then broadcast using the `sendrawtransaction` command, which submits it to the Bitcoin node and places it in the mempool.

```
Signed Transaction: {'hex': '02000000000101cd0e6f3d0264a0265493e7f72ba1d256e94729d439fe82a43c648b22ba03049b0000000017160014a88c61ad7515b65f6495f65c164f27f304
83b15fdffffff01f0a29a3b0000000017a914c7d6c7ca883c68ad718c32cece5125b9db4a8ae3870247304402202714 1be8c9fed630842c35abef64c1bb3ff6c25848858ce2327c315ff2e801bc02
07478b920b2feb81c729e2c01506e4990548d18aa9f31da05dac08f7e4630cda601210216105a3eeeb888b9d0cc62f10b9c7d29cb906ea4783baaa5fb9bc924ce315c1b00000000', 'complete':
True}

Transaction Broadcasted A' -> B'
TXID: 51c497d669bfdea4b5671f2b149723977cef66d60631cc55692f478b341ab35d

UTXO for B': 51c497d669bfdea4b5671f2b149723977cef66d60631cc55692f478b341ab35d
```

Since the experiment is performed in **regtest mode**, a new block is manually mined using the `generatetoaddress` command to confirm the transaction. After the block is mined, the transaction becomes part of the blockchain, and the funds are successfully transferred from **address A′ to address B′**. The output of this transaction then becomes a new **UTXO belonging to address B′**, which will be used in the next transaction step.

## Step 6: Create Transaction B′ → C′ :

The UTXO belonging to **B′** is retrieved and used as input for another transaction that sends funds to **C′**.

A new raw transaction is constructed with the `createrawtransaction` command. In this transaction, the UTXO belonging to B′ is used as the **input**, and a new output is created that sends the funds to **address C′**. Similar to the previous transaction, a small portion of the balance is deducted as a **transaction fee** to ensure the transaction can be processed by the network.

Once the raw transaction is created, it is decoded using the `decoderawtransaction` command. Decoding the transaction allows us to examine its structure, including the inputs, outputs, and associated scripts. In the decoded output, the **locking script (scriptPubKey)** for address C′ can be observed.

```
UTXO for B': 51c497d669bfdea4b5671f2b149723977cef66d60631cc55692f478b341ab35d

Raw Transaction B'->C': 02000000015db31a348b472f6955cc3106d666ef7c972397142b1f67b5a4debf69d697c4510000000000fdffffff01e07b9a3b0000000017a914116e49a58c06066c0f
ab9a2eda506fe92d26dddb8700000000

Decoded Transaction (Script Analysis):
 {'txid': 'f2ceee49f1000fc7a08f3eab13dc6383bd54733aeb95cff63a4a343eefdde445', 'hash': 'f2ceee49f1000fc7a08f3eab13dc6383bd54733aeb95cff63a4a343eefdde445', 'ver
sion': 2, 'size': 83, 'vsize': 83, 'weight': 332, 'locktime': 0, 'vin': [{'txid': '51c497d669bfdea4b5671f2b149723977cef66d60631cc55692f478b341ab35d', 'vout':
0, 'scriptSig': {'asm': '', 'hex': ''}, 'sequence': 4294967293}], 'vout': [{'value': Decimal('9.99980000'), 'n': 0, 'scriptPubKey': {'asm': 'OP_HASH160 116e49
a58c06066c0fab9a2eda506fe92d26dddb OP_EQUAL', 'desc': 'addr(2MtqPbU5BEoopfnJsUseTUsBrnHqM6qEgkE)#mj5q5q9q', 'hex': 'a914116e49a58c06066c0fab9a2eda506fe92d26dd
db87', 'address': '2MtqPbU5BEoopfnJsUseTUsBrnHqM6qEgkE', 'type': 'scripthash'}}]}
```

Since C′ is also a **P2SH-SegWit address**, the locking script follows the P2SH format, where the script contains a hash of the redeem script. This ensures that in order to spend the funds in the future, the spender must provide the correct redeem script and witness data that satisfy the conditions defined in the script.

## Step 7: Sign and Broadcast Transaction B′ → C′ :

The transaction is signed using the wallet keys through the `signrawtransactionwithwallet` command. This step generates the required digital signatures using the private key associated with address **B′**, proving the owner.

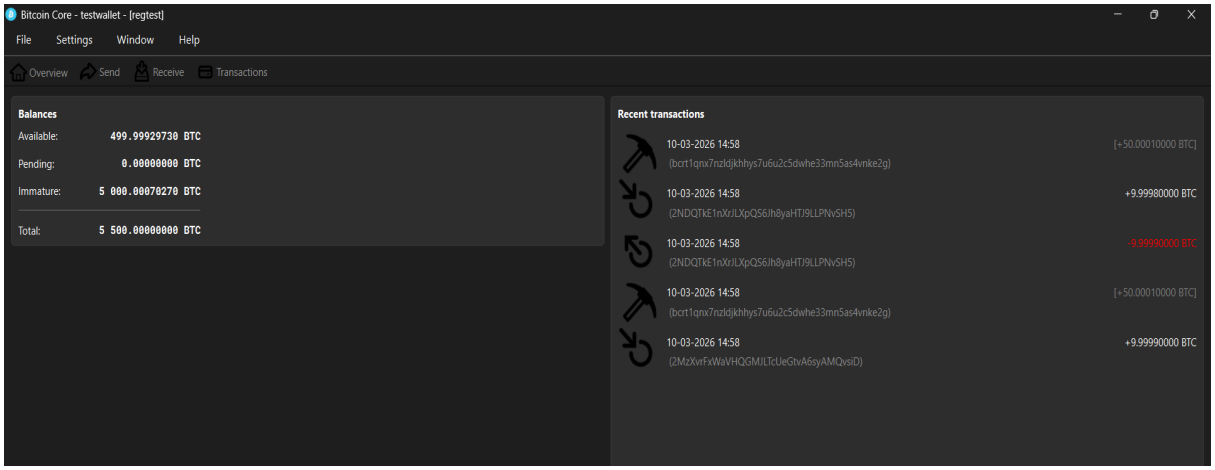
```
Signed Transaction: {'hex': '020000000001015db31a348b472f6955cc3106d666ef7c972397142b1f67b5a4debf69d697c4510000000017160014274a3f2fb4108c7f339703a571557d42bf0
90c40fdffffff01e07b9a3b0000000017a914116e49a58c06066c0fab9a2eda506fe92d26dddb87024730440220 6d79929382b0119eaec05862a0046316ac28550af2275c4636f8eba9c7669fc5022
00ecf03f1ca409ff2848cb87466ebc240f37be40aea38345224006467ef1eeb110121025f0d9e8a15badd05f9b9424e95535a8eee55a602f0d02cafc392345f80dd25f300000000', 'complete':
True}

Transaction Broadcasted B' -> C'
TXID: 66f994cdb80dd88b0401bf280db0c0d53c583078ac608f3be6a870420805c19c

SegWit transaction chain complete!
```

A block is mined to confirm the transaction, completing the SegWit transaction chain **A′ → B′ → C′**.

Once the block is generated, the transaction is permanently recorded on the blockchain, and the funds are successfully transferred to **address C′**. This completes the SegWit transaction chain **A′ → B′ → C′**, demonstrating how UTXOs are sequentially spent and transferred between addresses using P2SH-SegWit transactions.

## Bitcoin Core Wallet Transaction View :

The Bitcoin Core wallet interface confirms the successful execution of the SegWit transaction chain. The transaction history shows the transfer of funds from address A′ to B′ and then from B′ to C′. The wallet balance and mining rewards are also visible, indicating that blocks were generated to confirm the transactions in the regtest environment.




---

# Part 3: Analysis and Explanation

Below image displays a comparative analysis between a Legacy (P2PKH) transaction and a SegWit (P2SH-P2WPKH) transaction, highlighting the efficiency gains of the latter.

```
Enter TXID of Legacy P2PKH transaction (from Part 1): 35d94a4c3b07d62b49b4cc651043f311ac894ee4e485b31c016e330dd27b0492
Enter TXID of SegWit transaction (from Part 2): 66f994cdb80dd88b0401bf280db0c0d53c583078ac608f3be6a870420805c19c

==============================
Legacy P2PKH Transaction
==============================
TXID: 35d94a4c3b07d62b49b4cc651043f311ac894ee4e485b31c016e330dd27b0492
Size: 191 bytes
Virtual Size: 191 vbytes
Weight: 764

--- ScriptSig (Unlocking Script) ---
304402200512b387913ce26e455b27adb41b4430897d97400640da582d1653fab23f317b0220060e15c0e6d12062c3a3ad2c45101f76e80bb97edab360107bc29509a7679871[ALL] 03bfee9fcd44fe48969dd90846702d802e3d92296bd4164
b18b5e61

--- ScriptPubKey (Locking Script) ---
OP_DUP OP_HASH160 897734962996266cac2ca2e1af366a1d904cd22b OP_EQUALVERIFY OP_CHECKSIG

==============================
P2SH-P2WPKH SegWit Transaction
==============================
TXID: 66f994cdb80dd88b0401bf280db0c0d53c583078ac608f3be6a870420805c19c
Size: 215 bytes
Virtual Size: 134 vbytes
Weight: 533

--- scriptSig ---
0014274a3f2fb4108c7f339703a571557d42bf090c40

--- Witness Data ---
304402206d79929382b0119eaec05862a0046316ac28550af2275c4636f8eba9c7669fc502200ecf03f1ca409ff2848cb87466ebc240f37be40aea38345224006467ef1eeb1101
025f0d9e8a15badd05f9b9424e95535a8eee55a602f0d02cafc392345f80dd25f3

--- ScriptPubKey ---
OP_HASH160 116e49a58c06066c0fab9a2eda506fe92d26dddb OP_EQUAL

==============================
Transaction Size Comparison
==============================
P2PKH Transaction Size: 191 bytes
SegWit Transaction Size: 215 bytes
P2PKH Virtual Size: 191 vbytes
SegWit Virtual Size: 134 vbytes
P2PKH Weight: 764
SegWit Weight: 533

------------------------------
Result: SegWit transaction is smaller in virtual size.

SegWit separates signature data into the witness field.
Witness data counts less toward block weight, reducing transaction size and fees.
```

## 1.Data-Driven Size Comparison Table :

| Metric | Legacy (P2PKH) | SegWit (P2SH-P2WPKH) | Difference |
|---|---|---|---|
| **Total Size (Bytes)** | 191 B | 215 B | +24 Bytes |
| **Virtual Size (vSize)** | 191 vB | 134 vB | -57 vBytes |
| **Weight Units (WU)** | 764 WU | 533 WU | -231 WU |

## 2. Script Structure Comparison :

Difference in the Challenge-Response mechanism between the two parts:

- **P2PKH (Legacy):** The Response (Signature and Public Key) is stored inside the scriptSig field. This data is part of the transaction ID calculation and is expensive because it is weighted at 4 units per byte.
- **P2SH-P2WPKH (SegWit):** The scriptSig is nearly empty (containing only the P2SH redeem script). The actual Response is moved to a separate Witness Data field.

## 3. Why are SegWit Transactions Smaller?

- In Bitcoin, vSize is calculated as $vSize = Weight/4$
- Legacy data is weighted at **4 units per byte**.
- Witness data is weighted at only **1 unit per byte**.
- **Conclusion:** By moving the signature (the largest part of the transaction) into the Witness field, the network discounts that data, resulting in a much lower vSize despite the total raw size being slightly larger resulting in a much lower vSize. This effectively means that although a SegWit transaction takes up more physical

disk space, it consumes less of the 1MB 'base block' limit, allowing the network to process more transactions per block.

## 4. Benefits of SegWit

1. **Lower Transaction Fees:** Since fees are paid per vByte, SegWit transactions are ~30% cheaper for the user.
2. **Fixes Transaction Malleability:** Because the signature is moved to the Witness field (which is not part of the TXID calculation), the TXID cannot be changed by altering the signature.This is crucial for second-layer solutions like the Lightning Network, which rely on stable transaction IDs to build chains of unbroadcasted transactions
3. **Increased Capacity:** It allows more transactions to fit into a 1MB block without actually increasing the physical block size limit, effectively scaling the network.